

Guía de Estudio — Tema 10

Tipos de Datos y Sistemas de Tipos

Curso: Paradigmas y Lenguajes de Programación — IF009 **Institución:** Universidad Nacional de Tierra del Fuego — Instituto IDEI **Ciclo lectivo:** 2026 (1er semestre) **Módulo:** VII — Tipos de Datos **Tema:** 10 — Tipos de Datos y Sistemas de Tipos **Duración de la clase:** 360 minutos (clase unificada — Módulo VII completo) **Lenguaje principal:** TypeScript **Lenguajes de contraste:** C, Kotlin, Haskell, Python **Referencia bibliográfica principal:** Sebesta, *Concepts of Programming Languages* 12ª ed., Cap. 6 “Data Types” (pp. 259–324) **Referencias secundarias:** Louden & Lambert Cap. 8; Gabbrielli & Martini Cap. 8 **Filminas asociadas:** F-00 a F-53 (54 filminas)

1. Introducción al tema

¿Por qué existen los tipos de datos? Podríamos imaginar un lenguaje donde toda variable es “una caja” que guarda “algo” y el programador recuerda qué hay adentro. Ese lenguaje existe: se llama ensamblador, y programar en él es lento, propenso a errores y poco legible.

Los tipos de datos son la abstracción que permite al lenguaje —y al compilador— saber **qué valores** puede tomar una variable y **qué operaciones** tienen sentido sobre ella. Esta guía recorre, en el orden de la clase, las seis grandes familias de tipos (primitivos, ordinales, compuestos, referenciales, algebraicos y paramétricos) y luego sube un nivel: ¿qué es un **sistema de tipos**? ¿Cuándo dos tipos son “iguales”? ¿Qué significa que un lenguaje sea “fuertemente tipado”?

El hilo conductor es **TypeScript**, con contrastes puntuales en C, Kotlin, Haskell y Python. La clase completa corresponde al **Módulo VII** del plan mínimo y se dicta en una sesión unificada de 360 minutos dividida en cinco bloques.

 **Cómo usar esta guía:** cada sección referencia la filmina correspondiente ([F-XX]) para que puedas seguir la clase proyectada. Las citas bibliográficas (Sebesta §6.X , Louden §8.X , Gabbrielli §8.X) están respaldadas por la base de conocimiento ChromaDB y son verificables en las fuentes indicadas al final.

2. Objetivos de aprendizaje

Al finalizar el tema, el alumno podrá:

#	Objetivo	Nivel Bloom
OA1	Definir tipo de dato como conjunto de valores + conjunto de operaciones admisibles, y explicar por qué los lenguajes los usan (legibilidad, confiabilidad, seguridad)	Recordar/Comprender
OA2	Clasificar los tipos primitivos (numérico-entero, floating, boolean, char, decimal) y relacionar su representación interna con sus rangos y precisión	Comprender
OA3	Definir tipos ordinales por el usuario (enumeraciones, subrangos) y analizar cómo TypeScript, Kotlin y C los implementan con distintos niveles de seguridad	Analizar
OA4	Comparar arrays estáticos, semi-dinámicos y dinámicos según su binding time de forma y rango; calcular la función de acceso para arrays multidimensionales	Analizar
OA5	Explicar tipos secuencia (strings), su semántica de valor vs. referencia y las operaciones más comunes entre lenguajes	Comprender
OA6	Distinguir registros (structs) y tuplas de arrays; analizar implementación en memoria y casos de uso en TypeScript <code>interface / type</code> y Kotlin <code>data class</code>	Aplicar
OA7	Analizar uniones libres (C <code>union</code>) vs. uniones discriminadas (TypeScript, Haskell ADT) y razonar sobre seguridad de tipos; contrastar con Kotlin <code>sealed class</code>	Analizar
OA8	Explicar el tipo puntero: semántica, aritmética de punteros, punteros colgantes, memory leaks y garbage collection	Analizar
OA9	Aplicar tipos opcionales en TypeScript (<code>T null</code> , <code>?.</code> , <code>??</code> , <code>!</code>) con <code>strictNullChecks</code> ; comparar con Kotlin <code>T?</code> y Haskell <code>Maybe a</code>	Aplicar
OA10	Diferenciar equivalencia nominal vs. estructural de tipos y razonar sobre compatibilidad e incompatibilidad en sistemas tipados	Analizar
OA11	Comparar sistemas monomórficos vs. polimórficos; clasificar polimorfismo paramétrico (generics), ad-hoc (overloading) y por subtipo	Analizar
OA12	Evaluar el grado de fuertemente tipado (<i>strong typing</i>) en distintos lenguajes y relacionarlo con confiabilidad del software	Evaluar
OA13	Aplicar el chequeo de tipos (estático vs. dinámico) y explicar coerción implícita vs. conversión explícita	Aplicar
OA14	Distinguir tipos lista funcionales (Haskell, Python) de arrays y analizar sus operaciones head/tail/cons	Comprender

Fuente: `diseno.md` — Objetivos de Aprendizaje (OA1–OA14).

3. Conceptos previos necesarios

Esta guía **no re-explica** los temas de T09, pero los da por asumidos. Si alguno te resulta difuso, conviene repasar la guía de T09 antes de seguir.

Concepto previo	Tema	Por qué hace falta para T10
5-tupla de variables (nombre, dirección, tipo, valor, lifetime)	T09.1	Un “tipo” es uno de los componentes de la 5-tupla; hoy estudiamos ese componente en profundidad.
Binding de tipos (estático, dinámico, gradual)	T09.1	La clasificación de arrays por <i>binding time</i> de forma y rango (Bloque 2) reutiliza este vocabulario.
Scope y alias	T09.2	Las referencias (Bloque 3) son un tipo de alias; el scope determina la vida de las variables.
Garbage collection (reference counting, tracing GC)	T09.2	El Bloque 3 retoma el GC como solución a dangling pointers y memory leaks.
Tipado gradual	T09.1	TypeScript es gradual por construcción (<code>any</code>); aparece en strong typing (Bloque 4).

Conexión clave (filmina F-02): T09 respondió **CUÁNDO** se vincula un tipo a una variable. T10 responde **QUÉ** son los tipos como estructuras formales. No es repetición — es subir un nivel de abstracción.

4. Desarrollo teórico

Bloque 0 — Apertura y Conexión (15 min) · Filminas F-00 a F-03

La pregunta disparadora

La clase abre con una pregunta aparentemente simple **[F-01]**:

¿Qué diferencia hay entre el número `1` en C, en Python y en Haskell?

```
int x = 1;    // C: entero de 32 bits, complemento a 2, sin GC
```

```
x = 1        # Python: objeto int de precisión arbitraria, GC
```

```
x = 1        -- Haskell: Num a => a (polimórfico, determinado en uso)
```

Los tres imprimen `1`, pero son **objetos completamente distintos**: el de C son 4 bytes crudos con overflow silencioso; el de Python es un objeto en el heap con contador de referencias; el de Haskell es un valor polimórfico cuyo tipo concreto se decide según el uso. Esto muestra que los tipos no

son etiquetas: son **decisiones de diseño con consecuencias** en rendimiento, seguridad y expresividad.

Hoja de ruta [F-03]

Bloque	Contenido	Tiempo
1	Tipos Primitivos y Ordinales	70 min
2	Tipos de Agregación y Colecciones	90 min
3	Punteros, Null Safety, Tipos Recursivos	75 min
4	Sistemas de Tipos — Polimorfismo	80 min
5	Síntesis, Discusión y Cierre	30 min

Bloque 1 — Tipos Primitivos y Ordinales (70 min) · Filminas F-04 a F-14

4.1 ¿Qué es un tipo de dato? [F-04]

 **Definición formal (Sebesta §6.1):** “A data type defines a collection of data values and a set of predefined operations on those values” — Un tipo de dato define una **colección de valores** y un conjunto de **operaciones predefinidas** sobre esos valores. (Cita respaldada por ChromaDB, relevancia 0.627.)

Razones para incluir tipos en un lenguaje:

- **Legibilidad:** el código expresa la intención (`edad: number` dice más que `edad`).
- **Detectabilidad de errores:** el compilador o el runtime atrapan uso incorrecto antes de que ocurra en producción.
- **Reusabilidad:** abstraer comportamiento por tipo (polimorfismo, generics).

4.2 Clasificación canónica de tipos [F-05]

Familia	Ejemplos
Primitivos	<code>int</code> , <code>float</code> , <code>bool</code> , <code>char</code>
Definidos por el usuario	<code>enum</code> , <code>subrange</code> , <code>type alias</code> , <code>branded types</code>
Compuestos	<code>arrays</code> , <code>records</code> , <code>structs</code> , <code>tuples</code> , <code>maps</code> , <code>unions</code>
Referenciales	<code>pointers</code> , <code>references</code>
Algebraicos	productos, sumas, ADTs
Paramétricos	<code>generics</code> , <code>templates</code>

La clasificación no es estricta — muchos tipos cruzan categorías — pero sirve como mapa mental.

4.3 Tipos primitivos — preguntas guía [F-06]

Para estudiar cada tipo primitivo, hacerte cinco preguntas:

- 1. ¿Qué valores representa?
- 2. ¿Qué operaciones permite?
- 3. ¿Cómo se almacena?
- 4. ¿Qué errores puede detectar el compilador?
- 5. ¿Qué queda delegado al runtime?

Las familias son tres: **numéricos** (enteros, floating, decimal, complex), **lógicos** (`boolean`) y **texto elemental** (`char` / `string`).

4.4 Tipos numéricos — Enteros [F-07]

📖 **Sebesta §6.2.1.1 (Integer)**: los enteros se almacenan en **complemento a 2**. El bit más significativo es el signo: `0` = positivo, `1` = negativo. Por esto el rango no es simétrico: de **-2.147.483.648** a **+2.147.483.647** para 32 bits (2^{31} negativos, $2^{31}-1$ positivos — el cero ocupa un lugar del lado positivo). *(Cita respaldada por ChromaDB, relevancia 0.594.)*

Lenguaje	Tipo	Tamaño	Nota
C	<code>int</code> / <code>long</code>	32/64 bits	No hay GC; overflow silencioso; en C el overflow de <code>int</code> es undefined behavior
Java/Kotlin	<code>Int</code> / <code>Long</code>	32/64 bits	JVM; checked exceptions
TypeScript	<code>number</code>	64-bit IEEE 754	No distingue entero/float
TypeScript	<code>bigint</code>	Ilimitado	<code>9007199254740991n</code>
Python	<code>int</code>	Ilimitado	Objeto; GC

⚠️ **Atención TypeScript:** `number` es *siempre* un `double` IEEE 754 de 64 bits. Representa enteros exactos solo hasta `Number.MAX_SAFE_INTEGER` = $2^{53} - 1$. Para enteros más grandes existe `bigint` (literales con sufijo `n`).

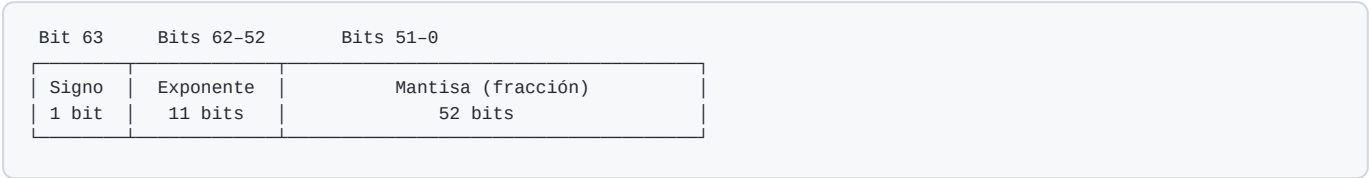
Representación en complemento a 2 (32 bits):

Valor	Binario (32 bits, complemento a 2)
+1	0000 0000 0000 0000 0000 0000 0000 0001
+42	0000 0000 0000 0000 0000 0000 0010 1010
-1	1111 1111 1111 1111 1111 1111 1111 1111
-42	1111 1111 1111 1111 1111 1111 1101 0110

4.5 Tipos numéricos — Floating Point [F-08]

 **Sebesta §6.2.1 (Floating-point types):** el estándar **IEEE 754** define la representación: signo + exponente + mantisa. *(Cita respaldada por ChromaDB, relevancia 0.593.)*

Estructura de un `double` (64 bits):



- **Signo:** `0` = positivo, `1` = negativo.
- **Exponente:** escala como potencia de 2 (desplazado en 1023).
- **Mantisa:** la parte fraccional — `1.mantisa × 2^(exponente-1023)` .

La trampa clásica: `0.1` en decimal no tiene representación finita en base 2 (es periódico, como 1/3 en decimal). La mantisa se redondea, y la suma hereda el error:

```
console.log(0.1 + 0.2) // 0.30000000000000004
console.log(0.1 + 0.2 === 0.3) // false ← ¡No usar == con floats!
```

Esto pasa en **cualquier lenguaje que use IEEE 754** (Python, Java, C, todos). Para aplicaciones financieras donde la exactitud decimal es crítica, TypeScript no tiene tipo decimal nativo — se usa la librería `decimal.js` . Contraste: Java/Kotlin tienen `BigDecimal` en el JDK; Python tiene `decimal.Decimal` en la biblioteca estándar.

4.6 Tipos numéricos especiales — Complejo y Decimal [F-09]

Tipo	Lenguaje	Soporte	Comentario
Complejo	Python	<code>complex</code> nativo (<code>3+4j</code>)	Primer ciudadano
Complejo	Fortran	Nativo	Orientado a cómputo científico
Complejo	Java/TS	Solo librería	Decisión: tipo vs. librería
Decimal	Python	<code>decimal.Decimal</code>	Módulo estándar
Decimal	Kotlin/Java	<code>BigDecimal</code>	En JDK
Decimal	TypeScript	<code>decimal.js</code>	Librería externa

Idea central: un lenguaje decide qué tipos incluye en su núcleo según sus prioridades: eficiencia, simplicidad, seguridad, precisión o dominio de aplicación.

4.7 Boolean y Char [F-10]

 **Sebesta §6.2.4 (Boolean types):** “Boolean types are perhaps the simplest of all types. [...] One popular exception is C89, in which numeric expressions can be used as if they were Boolean” — En C89 no existe tipo boolean: cualquier entero distinto de 0 es “verdadero”. (Cita respaldada por ChromaDB, relevancia 0.627.)

Lenguaje	Boolean	Char	Observación
TypeScript	<code>boolean</code> real	no existe	<code>"a"</code> es string de longitud 1
C (clásico)	<code>0</code> /no-cero	<code>char</code> (byte)	Sin tipo propio
Kotlin	<code>Boolean</code>	<code>Char</code> UTF-16	Tipos reales
Python	<code>bool</code> (subtipo de <code>int</code> !)	<code>str</code> de longitud 1	<code>True == 1</code> da <code>True</code>

 **Dato curioso Python:** `bool` es subtipo de `int`. `True == 1` da `True`. Eso hereda de la decisión de Python de hacer todo objeto y de la transición histórica desde C sin boolean.

4.8 Tipos ordinales y dominios finitos [F-11]

Un tipo **ordinal** tiene valores discretos que pueden enumerarse y, en muchos lenguajes, ordenarse. Ejemplos: `boolean`, `char`, `integer`, `enum`, `subrange`.

Propiedades: valores discretos, sucesor/predecesor, comparación por orden, posibilidad de definir rangos válidos.

Idea central: los tipos ordinales permiten que el sistema de tipos modele dominios finitos o restringidos. En vez de aceptar cualquier entero para un día del mes, puedo restringir a `1..31`.

4.9 Enumeraciones en TypeScript [F-12]

```
// Numeric enum
enum Direction { Up, Down, Left, Right }
// Up = 0, Down = 1, Left = 2, Right = 3

const move = (dir: Direction) => {
  if (dir === Direction.Up) console.log("subiendo")
}
move(Direction.Up)

// String enum – para interoperabilidad con APIs
enum Status { Active = 'active', Inactive = 'inactive' }
const s: Status = Status.Active // valor en runtime: 'active'

// const enum – inlining en compilación (sin objeto en runtime)
const enum Color { Red = 0, Green = 1, Blue = 2 }
// El compilador reemplaza Color.Red por 0 directamente
```

Una enumeración crea un **dominio simbólico cerrado**. TypeScript las trata como tipo de primera clase — no se puede pasar `42` donde se espera `Direction`.

4.10 Enumeraciones — comparación multilenguaje [F-13]

📖 **Sebesta §6.4.1 (Enumeration Types):** el problema con `enum` en C es que **no son type-safe** — pueden ser mezclados con enteros sin advertencia del compilador. (Cita respaldada por ChromaDB, relevancia 0.657.)

Aspecto	C <code>enum</code>	TypeScript <code>enum</code>	Kotlin <code>enum class</code>	Haskell <code>data</code>
Tipo propio	✗	✓	✓	✓
Type-safe	✗ (es un <code>int</code>)	✓	✓	✓
Propiedades	✗	Limitado	✓	✗ (ADT)
Métodos	✗	✗	✓	✗ (funciones)
Exhaustividad	No	Con <code>never</code>	<code>when</code> garantiza	Pattern matching
Ejemplo	<pre>enum Dir { UP };</pre>	<pre>enum Dir { Up };</pre>	<pre>enum class Dir { UP };</pre>	<pre>data Dir = Up</pre>

Kotlin va en la dirección opuesta a C: `enum class` puede tener propiedades y métodos. Haskell va más lejos: `data Color = Red | Green | Blue` es un ADT completo con pattern matching exhaustivo.

4.11 Rangos y subrangos [F-14]

- **Rango:** porción contigua de valores dentro de un tipo ordinal (`1..10` , `'a'..'z'`).
- **Subrango:** un **tipo** cuyo dominio queda restringido a un rango de otro tipo ordinal.

Pascal y Ada tienen subrangos nativos: `type Nota = 0..10` crea un tipo real; el compilador rechaza asignar `15` a una variable `Nota` .

Python tiene `range(0, 11)` que representa los valores 0 a 10, pero **no crea un tipo**. `nota = 15` sigue siendo válido. Para modelar un subrango hay que programar la validación explícita:

```
class Nota:
    def __init__(self, valor: int):
        if valor < 0 or valor > 10:
            raise ValueError("Nota inválida")
        self.valor = valor
```

📖 **Louden §8.2:** “Languages in the C family (C, C++, Java) do not have subrange types, since the same effect can be achieved using enumerated types” — (Cita respaldada por ChromaDB, relevancia 0.503.) Gabbrielli §8.3.9 (Intervals) — relevancia 0.529.

TypeScript lo simula con literal unions: `type Day = 1|2|3|4|5` . El compilador **sí** rechaza `const d: Day = 6` . No es un subrango en el sentido de Pascal, pero logra el mismo nivel de seguridad estática.

Bloque 2 — Tipos de Agregación y Colecciones (90 min) · Filminas F-15 a F-30

4.12 Tipos compuestos — formas principales [F-15]

Un tipo compuesto construye valores más grandes a partir de otros tipos. Hay cuatro formas algebraicas principales:

Forma	Significado	Ejemplos
Producto	varios campos simultáneos	<code>record</code> , <code>struct</code> , <code>tuple</code> , <code>object</code>
Secuencia	colección ordenada de elementos	<code>array</code> , <code>list</code> , <code>string</code>
Mapeo	asociación clave → valor	<code>map</code> , <code>dictionary</code> , <code>record</code> dinámico
Suma / unión	una alternativa entre varias formas	<code>union</code> , <code>discriminated union</code> , <code>sealed class</code> , <code>ADT</code>

4.13 Strings — secuencias de texto [F-16]

📖 **Sebesta §6.3 (Character String Types):** “A character string type is one in which the values consist of sequences of characters” — Las operaciones más comunes son: **asignación, concatenación, referencia a subcadenas, comparación y coincidencia de patrones**. (Cita respaldada por ChromaDB, relevancia 0.67.)

Lenguaje	Naturaleza	Mutabilidad	Comparación
TypeScript	Primitivo (con métodos via boxing)	Inmutable	<code>===</code> por valor ✓
Kotlin	Objeto (<code>String</code>)	Inmutable	<code>==</code> por valor (llama <code>equals</code>)
Java	Objeto (<code>String</code>)	Inmutable	<code>==</code> por referencia ← ERROR clásico
C	Puntero a <code>char[]</code> terminado en <code>\0</code>	Mutable	<code>strcmp()</code> (no <code>==</code>)
Python	Objeto <code>str</code>	Inmutable	<code>==</code> por valor ✓

⚠️ **Error clásico en Java:** `str1 == str2` compara **referencias**, no contenido. Hay que usar `.equals()`. En TypeScript, `===` compara por **valor** siempre — no hay sorpresas. TypeScript hereda de JavaScript: `string` es primitivo pero el runtime hace autoboxing para permitir llamar métodos como `.toUpperCase()`. El resultado siempre es una nueva string — son inmutables.

4.14 Arrays — taxonomía por binding time [F-17]

📖 **Sebesta §6.5 (Array Types, Figura 6.2):** la clasificación es por **binding time de forma** y **binding time de rango**. (Cita respaldada por ChromaDB, relevancia 0.492.)

Categoría	Forma	Rango	Almacenamiento	Ejemplo
Estático	Estático	Compila	Estático	<code>int a[10]</code> en C (global)
Semi-dinámico (stack)	Fija en runtime	Post-elaboración	Stack (se libera al salir)	<code>int a[n]</code> en C99 (local)
Heap fijo	Dinámica (heap)	Fija luego	Heap (manual)	<code>new Array<number>(n)</code> en TS
Heap flexible	Totalmente dinámica	Cambia	Heap (GC)	<code>number[]</code> en TS / <code>list</code> Python

En TypeScript, el tipo que usamos todo el tiempo — `T[]` — es un array de **heap flexible**. El motor gestiona el realloc automáticamente.

4.15 Arrays — función de acceso multidimensional [F-18]

- **Row-major** (C, Java, Kotlin, Python): los elementos de una **fila** se almacenan contiguos.

```
a[i][j] → base + (i × cols + j) × size
```

- **Column-major** (Fortran, MATLAB, R): los elementos de una **columna** se almacenan contiguos.

```
a[i][j] → base + (i + j × rows) × size
```

¿Por qué importa? Cache locality. Acceder en el orden incorrecto genera cache misses → lentitud. Multiplicación de matrices en C iterando en row-major order puede ser hasta **10× más rápido** que iterar en el orden opuesto.

```
// Row-major access — eficiente
for (let i = 0; i < n; i++)
  for (let j = 0; j < m; j++)
    suma += matrix[i][j] // accede fila a fila ✓
```

4.16 Arrays — rectangulares vs. jagged + slices [F-19]

- **Rectangular** (Fortran, C#): todos los sub-arrays tienen **la misma longitud**. `int[, ]` en C#: acceso más rápido y lineal en memoria.
- **Jagged** (C, Java, Kotlin): array de **punteros a arrays** — cada fila puede tener distinto tamaño. Java: **todos** los arrays multidimensionales son jagged por diseño. Mayor flexibilidad; mayor overhead por indirección.

Slices — referencia sin copia:

Lenguaje	Sintaxis	Comportamiento
Python	<code>lista[1:4]</code>	Produce nueva lista (copia)
Kotlin	<code>lista.subList(1, 4)</code>	Vista viva — comparte memoria
Go	<code>a[1:4]</code>	Ciudadano de primera clase (length + capacity)
Ada	<code>A(2..4)</code>	Slice de array nativo

4.17 Arrays asociativos — Maps [F-20]

 **Sebesta §6.8 (Associative Arrays):** “An associative array is an unordered collection of data elements that are indexed by an equal number of values called keys” — (Cita respaldada por ChromaDB, relevancia 0.489.)

```
// Map tipado
const scores = new Map<string, number>([["Ana", 9], ["Luis", 7]])
scores.set("Marta", 8)
console.log(scores.get("Ana")) // 9
scores.has("Luis") // true
scores.delete("Luis")

// Record — objeto como mapa de clave string
const config: Record<string, boolean> = { dark: true, sound: false }
config["alerts"] = true
```

Contraste Kotlin:

```
val scores = mutableMapOf("Ana" to 9, "Luis" to 7)
scores["Marta"] = 8
println(scores["Ana"]) // 9
```

TypeScript: `Map<K, V>` para tipos arbitrarios de clave · `Record<string, V>` para claves string.

4.18 Arrays en TypeScript — cuatro formas [F-21]

```
// 1. Array dinámico: heap flexible
const lista: number[] = [1, 2, 3];
lista.push(4);

// 2. Array con tamaño inicial, pero sigue siendo dinámico
const fijo: number[] = new Array(5).fill(0).map((_, i) => i * 2);
fijo.push(10); // permitido

// 3. TypedArray: tamaño fijo y memoria binaria
const binario: Int32Array = new Int32Array([10, 20, 30]);
// binario.push(40); // error: no existe push

// 4. ReadonlyArray: solo lectura a nivel de TypeScript
const constante: ReadonlyArray<number> = [1, 2, 3];
// constante.push(4); // error de compilación
```

4.19 TypedArrays — almacenamiento binario directo [F-22]

¿Por qué `Int32Array` no es lo mismo que `number[]` ? En JavaScript/TypeScript, cada `number` es siempre un `double` IEEE 754 de 64 bits, envuelto en un objeto del motor V8. Eso es costoso

(overhead de boxing).

Un `TypedArray` como `Int32Array` almacena enteros en **binario puro**, 4 bytes por elemento, en un `ArrayBuffer` — bloque de bytes crudos en memoria. Sin boxing, sin overhead.

```
const binario: Int32Array = new Int32Array([10, 20, 30])
// Internamente: un ArrayBuffer de 12 bytes (3 × 4 bytes), sin boxing
// En memoria: [0A 00 00 00 | 14 00 00 00 | 1E 00 00 00] (little-endian)
```

Tipo	Bits	Rango	Equivalente C
<code>Int8Array</code>	8	−128 a +127	<code>int8_t</code>
<code>Uint8Array</code>	8	0 a 255	<code>uint8_t</code>
<code>Int16Array</code>	16	−32.768 a +32.767	<code>int16_t</code>
<code>Int32Array</code>	32	−2.147.483.648 a +2.147.483.647	<code>int32_t</code>
<code>Uint32Array</code>	32	0 a 4.294.967.295	<code>uint32_t</code>
<code>Float32Array</code>	32	$\pm 3.4 \times 10^{38}$ (precisión simple)	<code>float</code>
<code>Float64Array</code>	64	$\pm 1.8 \times 10^{308}$ — igual que <code>number</code>	<code>double</code>

¿Cuándo usar `TypedArrays`? Procesamiento de imagen (`Uint8ClampedArray` para píxeles en Canvas), WebGL/shaders (la GPU requiere tipos binarios exactos), comunicación binaria (WebSocket, FileReader, protocolos de red), algoritmos numéricos de alto rendimiento (memoria contigua = menos cache misses).

4.20 Registros — `interface` y `type` en TypeScript [F-23]

📖 **Sebesta §6.7 (Record Types):** “Records are frequently valuable data types in programming languages. The design of record types is straightforward, and their use is safe” — “Records and arrays are closely related structural forms”. (Cita respaldada por ChromaDB, relevancia 0.492.)

```
// interface — preferida para objetos extensibles
interface Usuario {
  nombre: string
  email: string
  edad: number
}

// type — equivalente funcional
type Punto = { x: number; y: number }

// Inmutabilidad con readonly
type PuntoFijo = { readonly x: number; readonly y: number }

// Tipo con campo opcional
interface Config { debug: boolean; nivel?: number }
```

Contraste Kotlin `data class` :

```
data class Usuario(val nombre: String, val email: String, val edad: Int)
// Genera: equals, hashCode, toString, copy() automáticos
// TypeScript no tiene equivalente nativo — se implementa manualmente
```

4.21 Tuplas — producto cartesiano formal [F-24]

📖 **Sebesta §6.8 (Tuple Types):** “A tuple is a data type that is similar to a record, except that the elements are not named” — (Cita respaldada por ChromaDB, relevancia 0.458.)

Tipo $A \times B$ = todos los pares (a, b) con $a: A$, $b: B$.

```
const par: [string, number] = ["hola", 42]

// Desestructuración
const [nombre, edad] = par
console.log(nombre) // "hola"

// Tipo nombrado
type RGB = [number, number, number]
const rojo: RGB = [255, 0, 0]

// Tupla con elemento opcional (TypeScript 3+)
type HttpResponse = [number, string, string?]
```

Kotlin tiene `Pair<A,B>` y `Triple<A,B,C>` — solo hasta 3 elementos nativos. Python tiene tuplas inmutables (a, b, c) por defecto. Haskell tiene tuplas como tipos algebraicos nativos.

4.22 Tipos algebraicos de datos en Haskell [F-25]

Un ADT permite definir un nuevo tipo a partir de un **conjunto cerrado de variantes** posibles.

```
data Resultado
  = Exito String
  | Error String
  | Cargando

mostrar :: Resultado -> String
mostrar resultado =
  case resultado of
    Exito valor    -> "Valor recibido: " ++ valor
    Error mensaje -> "Error: " ++ mensaje
    Cargando       -> "Cargando..."
```

Cada variante es un constructor distinto. El `case` pattern matching es **exhaustivo** — si falta un caso, el compilador avisa. Esto es la base conceptual de las uniones discriminadas que veremos en TypeScript.

4.23 Listas funcionales — definición y multilenguaje [F-26]

📖 **Sebesta §6.9 (List Types):** “Lists are collections of data that can have varying numbers of elements and to which elements can be easily added or removed from either end” — (Cita respaldada por ChromaDB, relevancia 0.492.)

Origen: LISP (1958). **Propiedades fundamentales:**

- **head:** primer elemento
- **tail:** resto de la lista
- **cons:** construcción añadiendo al frente
- **Recursividad inherente:** lista = head + tail (otra lista)

```
xs = [1, 2, 3, 4]
head xs    -- 1
tail xs    -- [2, 3, 4]
1 : xs     -- [1, 1, 2, 3, 4]
```

Diferencia clave con arrays: acceso aleatorio en lista enlazada es $O(n)$ — hay que recorrer desde el inicio. Arrays tienen $O(1)$. Pero inserción al frente de lista enlazada es $O(1)$ — arrays requieren $O(n)$ para desplazar.

Lenguaje	Tipo	Estructura real	Inmutable
TypeScript	<code>T[]</code> / <code>Array<T></code>	Array dinámico	✗
TypeScript	<code>ReadonlyArray<T></code>	Array dinámico	✓
Kotlin	<code>List<T></code>	ArrayList	✓
Kotlin	<code>MutableList<T></code>	ArrayList	✗
Python	<code>list</code>	Array dinámico (¡no lista enlazada!)	✗
Haskell	<code>[a]</code>	Lista enlazada real	✓ siempre

⚠ **Dato importante:** Python `list` es en realidad un array dinámico, no una lista enlazada. Tiene acceso $O(1)$ por índice. Haskell `[a]` sí es una lista enlazada real, inmutable, con pattern matching.

4.24 Tipos algebraicos: producto vs. suma [F-27]

- **Tipo producto** — un valor contiene A **y** B simultáneamente:

```
type Punto = { x: number; y: number }
// Punto = number × number
```

- **Tipo suma** — un valor puede ser A **o** B, discriminado por una etiqueta:

```
type Resultado =
  | { kind: "ok";      value: number }
  | { kind: "error";   message: string }
// Resultado = Ok + Error
```

Idea central: producto = **combinación** (todos los campos); suma = **alternativa** (uno de los campos, discriminado).

4.25 Uniones libres (unsafe) — C `union` [F-28]

📖 **Sebesta §6.10 (Union Types):** “Type checking of unions requires that each union construct include a type indicator. Such an indicator is called a tag, or discriminant, and a union with a discriminant is called a discriminated union. The first language to provide discriminated unions was ALGOL 68” — (Cita respaldada por ChromaDB, relevancia 0.691.)

```
union Data {
  int i;
  float f;
  char c;
};

union Data d;
d.i = 42;
printf("%f\n", d.f); // ← Lee basura — interpretación incorrecta del bit pattern
```

¿Qué es un “bit pattern”? Cuando se escribe `d.i = 42`, los 4 bytes de la unión toman el patrón binario del entero 42:

42 como int32 (complemento a 2):

00000000	00000000	00000000	00101010
----------	----------	----------	----------

← d.i = 42

Al leer `d.f`, esos mismos 4 bytes se **reinterpretan** como un float IEEE 754 — sin conversión, solo se leen los mismos bits con otra semántica. El resultado es **basura aritmética** y el programa continúa sin lanzar ningún error.

Este es el peligro de las uniones en C: el compilador no sabe qué campo está “activo”. Las uniones discriminadas (TypeScript `kind`, Kotlin `sealed`) resuelven esto al nivel del sistema de tipos.

4.26 Uniones discriminadas (safe) — TypeScript [F-29]

```
type Result<T> =
  | { kind: 'success'; value: T }
  | { kind: 'error'; message: string }

function handle(r: Result<number>) {
  switch (r.kind) {
    case 'success':
      console.log(r.value) // TypeScript sabe: r es { kind: 'success'; value: number }
      break
    case 'error':
      console.log(r.message) // TypeScript sabe: r es { kind: 'error'; message: string }
      break
  }
}
```

Seguridad de tipos:

- El campo `kind` actúa como **etiqueta discriminante**.
- TypeScript **narrowea** el tipo en cada rama del switch.
- Con `--strictNullChecks`, si hay un caso sin manejar: error de compilación.

4.27 Uniones discriminadas — TypeScript vs. Kotlin vs. Haskell [F-30]

 **Gabbrielli §8.4.3 (Tagged Unions):** “A tagged union is the union of several other types, where, however, each value maintains trace of the original type it comes from. It is the modern evolution of Algol 68’s unions” — (Cita respaldada por ChromaDB, relevancia 0.682.)

```
sealed class Result<out T> {
    data class Success<T>(val value: T)           : Result<T>()
    data class Error(val message: String)         : Result<Nothing>()
}

fun handle(r: Result<Int>) = when (r) { // exhaustividad GARANTIZADA por el compilador
    is Result.Success -> println(r.value)
    is Result.Error   -> println(r.message)
}
```

```
data Result a = Success a | Error String

handle :: Result Int -> IO ()
handle (Success v) = print v
handle (Error msg) = putStrLn msg
```

Aspecto	TypeScript	Kotlin	Haskell
Mecanismo	kind explícito	sealed class	ADT (data)
Exhaustividad	Con never trick	Compilador garantiza	Pattern matching
Runtime tag	Campo de objeto	is check	Constructor tag

La diferencia es que TypeScript requiere esfuerzo explícito (el never trick o la opción del compilador); Kotlin y Haskell lo hacen por diseño.

Bloque 3 — Punteros, Null Safety y Tipos Recursivos (75 min) · Filminas F-31 a F-39

4.28 Tipo puntero — definición y semántica [F-31]

 **Sebesta §6.11.1 (Pointer Types):** “A pointer type variable has a range of values that consists of memory addresses and a special value, nil” — (Cita respaldada por ChromaDB, relevancia 0.699.)

```
int x = 42;
int* ptr = &x; // & → toma la dirección de x
int y = *ptr;  // * → dereferencia: accede al valor en la dirección
ptr++;         // aritmética de punteros – avanza sizeof(int) bytes
```

Usos legítimos de punteros:

- Manejo indirecto de datos en el heap
- Paso de estructuras grandes por referencia (eficiencia)
- Construcción de estructuras dinámicas (listas enlazadas, árboles)

4.29 Punteros — problemas clásicos [F-32]

📖 **Sebesta §6.11.6 (Dangling pointers) + Gabbrielli §8.5 (Dangling references):** “If a language allows dangling references to happen, it is obvious that it cannot be type safe” — (Citas respaldadas por ChromaDB, relevancia 0.688 y 0.735 respectivamente.)

Problema	Qué pasa	Lenguaje afectado
Dangling pointer	Puntero apunta a memoria ya liberada	C, C++
Memory leak	Memoria asignada sin referencia, sin liberar	C, C++
Double free	Liberar el mismo bloque dos veces → UB	C
Null dereference	*null → crash	Java, C, pre-Kotlin, pre-TS strict
Buffer overflow	Aritmética fuera de rango → sobreescritura	C

Ejemplos:

- **Dangling pointer:** `free(ptr); *ptr = 5;` — ya liberaste esa memoria; otro objeto puede estar ahí.
- **Memory leak:** olvidar llamar `free()` — la memoria nunca vuelve al sistema. En procesos largos, crece indefinidamente.
- **Double free:** llamar `free(ptr)` dos veces — comportamiento indefinido, puede corromper el heap.
- **Null dereference:** `*NULL` → crash con SIGSEGV.
- **Buffer overflow:** `int a[5]; a[10] = 0;` — sobreescribe memoria adyacente. Causa de vulnerabilidades de seguridad históricas.

Estos cinco problemas son la **principal motivación histórica** para los lenguajes con GC y los sistemas de tipos modernos.

4.30 Soluciones históricas — Tombstones y Locks [F-33]

📖 **Sebesta §6.11.4** documenta dos soluciones históricas:

- **Tombstones:** cada objeto heap tiene un tombstone (marcador). Al liberar, el tombstone se marca como “liberado”. Los punteros apuntan al tombstone, no al objeto directo. Al acceder: si el tombstone está marcado → error en lugar de basura.
- **Locks and Keys:** cada puntero lleva una clave. Cada bloque de heap tiene un lock. Al acceder: se verifica que la clave coincida con el lock del bloque. Si no coinciden → acceso denegado (error detectable).

Soluciones modernas:

- **Garbage Collection** (Java, Kotlin, JS/TS, Python, Go) — elimina dangling y leaks automáticamente.
- **Ownership + Borrow checker** (Rust) — verificación en compilación sin GC.

Las soluciones históricas tienen overhead de memoria y rendimiento. El GC las reemplazó en la mayoría de los lenguajes modernos.

4.31 Referencias vs. Punteros [F-34]

📖 **Sebesta §6.11.5 (Reference Types):** “A reference variable is a constant pointer that is always implicitly dereferenced” — (Cita respaldada por ChromaDB, relevancia 0.68.)

Aspecto	Puntero ©	Referencia (C++)	Referencias JS/TS/Kotlin
Aritmética	✅ <code>ptr++</code>	❌ No permitida	❌ No existe
Reassignable	✅	❌ Siempre apunta al mismo objeto	✅ Variables reassignables
Puede ser null	✅ <code>NULL</code>	❌ Debe inicializarse	✅ (con nullable)
Derreferencia explícita	✅ <code>*ptr</code>	❌ Implícita	❌ Implícita
Gestión de memoria	Manual	Manual (o RAII)	GC automático

TypeScript / JavaScript: sin punteros explícitos — todo objeto vive en heap, gestionado por V8. Las variables almacenan **referencias implícitas** (igual que Java/Kotlin). El GC se encarga de todo — sin `free()`, sin `delete`.

4.32 Tipos recursivos — definición [F-35]

Un tipo **recursivo** se define en términos de sí mismo.

```
struct Node {
  int      value;
  struct Node* next; // referencia recursiva – posible por ser puntero
};

data List a = Nil | Cons a (List a)
-- Isomorfo al tipo built-in [a]
```

Clave: la recursión es posible porque el “campo recursivo” es una **referencia/puntero** (tamaño fijo), no el objeto completo (que sería infinito). Si fuera `struct Node { int value; struct Node next; }` — sin puntero — el compilador calcularía un tamaño infinito.

4.33 Tipos recursivos — árbol binario en TypeScript [F-36]

```
type BinaryTree<T> =  
  | { kind: 'leaf' }  
  | { kind: 'node'; value: T; left: BinaryTree<T>; right: BinaryTree<T> }  
  
// Función que explota el tipo recursivo  
function depth<T>(t: BinaryTree<T>): number {  
  if (t.kind === 'leaf') return 0  
  return 1 + Math.max(depth(t.left), depth(t.right))  
}  
  
const arbol: BinaryTree<number> = {  
  kind: 'node', value: 1,  
  left: { kind: 'node', value: 2, left: { kind: 'leaf' }, right: { kind: 'leaf' } },  
  right: { kind: 'leaf' }  
}
```

Contraste Kotlin `sealed class` :

```
sealed class Tree<T> {  
  object Empty : Tree<Nothing>()  
  data class Node<T>(val value: T, val left: Tree<T>, val right: Tree<T>) : Tree<T>()  
}
```

Relación clave: tipos recursivos + uniones discriminadas = **estructuras inductivas**. Esto es la base de Haskell, Kotlin `sealed` y TypeScript.

4.34 El problema del null — “Mi error de un billón de dólares” [F-37]

📖 **Sebesta §6.12 (Optional Types):** “There are situations in programming when there is a need to be able to indicate that a variable does not currently have a value. Some older languages use zero as a nonvalue for numeric variables. This approach has the disadvantage of not being able to distinguish between when the variable is supposed to have the value zero and when it has no value” — (Cita respaldada por ChromaDB, relevancia 0.613.)

Tony Hoare — inventor del null pointer (1965, ALGOL W):

“I call it my billion-dollar mistake. [...] It has led to innumerable errors, vulnerabilities, and system crashes” — QCon 2009

El problema: en Java (y C, Python sin disciplina), **cualquier referencia puede ser `null`**. No hay distinción de tipos entre “una string válida” y `null`. El error ocurre en **runtime** — el compilador no puede ayudar.

```
String nombre = getUser().getName(); // puede retornar null  
nombre.toUpperCase();                 // NullPointerException en runtime
```

La solución: hacer null parte del sistema de tipos.

- Si `null` es un valor posible → el tipo debe decirlo explícitamente.
- Si el tipo no incluye `null` → el compilador garantiza que nunca es null.

4.35 TypeScript Null Safety — operadores y código [F-38]

Con `"strictNullChecks": true` en `tsconfig.json`:

Operador	Nombre	Comportamiento
<code>T null</code>	Tipo nutable	Declara que <code>T</code> puede ser null
<code>T undefined</code>	Tipo indefinido	Parámetros opcionales por defecto
<code>x?.prop</code>	Optional chaining	Si <code>x</code> es null/undefined → retorna <code>undefined</code>
<code>x ?? valor</code>	Nullish coalescing	Si <code>x</code> es null/undefined → usa <code>valor</code>
<code>x!</code>	Non-null assertion	Fuerza acceso — puede fallar en runtime
<code>if (x !== null)</code>	Type narrowing	El compilador infiere tipo más estrecho
<code>function f(x?: T)</code>	Parámetro opcional	<code>x</code> es <code>T undefined</code>

```
interface User { name: string; email: string | null }

function sendEmail(user: User) {
  // Nullish coalescing: usa "sin email" si email es null/undefined
  const dest = user.email ?? 'sin email'

  // Optional chaining: retorna undefined si email es null (no lanza error)
  const upper = user.email?.toUpperCase()

  // Type narrowing: en este bloque, TypeScript sabe que email es string
  if (user.email !== null) {
    console.log(`Enviando a ${user.email.toUpperCase()}`)
  }

  // Non-null assertion: NO usar salvo que estés seguro
  // const forced = user.email!.toUpperCase() ← peligroso
}
```

Con `strict: true`, el compilador **rechaza** acceder a `user.email.toUpperCase()` directamente — te fuerza a verificar primero. Eso es exactamente lo que Tony Hoare pidió 40 años tarde.

4.36 Null Safety — comparación multilenguaje [F-39]

Lenguaje	Mecanismo	Activación	Safety level
C / Java (pre-8)	<code>null</code> sin restricción	Siempre	✗ Peligroso
Python	<code>None</code> sin restricción	Siempre	✗ Peligroso
TypeScript	<code>T null</code> + <code>strictNullChecks</code>	Opt-in	✓ (config-dependiente)
Kotlin	<code>T</code> vs <code>T?</code> + operadores <code>?.</code> <code>?:</code> <code>!!</code>	Siempre	✓ Type-safe por diseño
Haskell	<code>Maybe a = Nothing Just a</code>	Siempre	✓ Type-safe (monádico)
Rust	<code>Option<T> = None Some(T)</code>	Siempre	✓ Type-safe

Comparación de operadores equivalentes:

TypeScript	Kotlin	Significado
<code>x ?? y</code>	<code>x ?: y</code>	Si null → usar y (Elvis)
<code>x?.prop</code>	<code>x?.prop</code>	Acceso seguro (igual sintaxis)
<code>x!</code>	<code>x!!</code>	Forzar no-null (peligroso)
<code>if (x !== null)</code>	<code>if (x != null)</code>	Type narrowing

Conexión con T02 (mónadas): `Maybe` / `Option` es la mónada de null safety — `flatMap = ?.` encadenado. El `?.` en TypeScript es esencialmente `bind/flatMap` sobre `T | null`.

Bloque 4 — Sistemas de Tipos: Monomórficos, Polimórficos y Strong Typing (80 min) · Filminas F-40 a F-48

4.37 Sistemas monomórficos [F-40]

Cada expresión tiene **exactamente un tipo**.

```
int max_int(int a, int b) { return a > b ? a : b; }
float max_float(float a, float b) { return a > b ? a : b; }
// Misma lógica, dos funciones – código duplicado inevitable
```

Ventajas: simple y predecible, herramientas de análisis directas, sin overhead de polimorfismo.

Limitación: código duplicado. `max(int, int)` y `max(float, float)` requieren funciones separadas. No escala a bibliotecas grandes. **Motivación histórica** para introducir el polimorfismo.

4.38 Polimorfismo — taxonomía [F-41]

📖 **Louden §8.8 (Polymorphic Type Checking)** — relevancia ChromaDB 0.767. **Gabbrielli §8.8:** “We introduced the concept of polymorphism... where we also distinguished between two radically different forms of it: overloading (or ad hoc polymorphism) and universal polymorphism” — relevancia 0.726.

Taxonomía de Strachey (1967) y Cardelli & Wegner (1985):

- **Polimorfismo Ad-hoc** (apariencia de uniformidad):
 - **Sobrecarga (overloading):** mismo nombre, múltiples implementaciones por tipo.
 - **Coerción:** conversión implícita que permite que un tipo sea tratado como otro.
- **Polimorfismo Universal** (uniformidad real):
 - **Paramétrico (generics):** una implementación para todos los tipos — el tipo es un parámetro.
 - **Subtipo / Inclusión:** S puede usarse donde se espera T si S es subtipo de T.

Diferencia clave: ad-hoc = apariencia de uniformidad; paramétrico = uniformidad real.

4.39 Polimorfismo ad-hoc — sobrecarga en TypeScript [F-42]

```
// Firmas declaradas (interfaz visible al consumidor)
function area(radio: number): number
function area(base: number, altura: number): number

// Implementación única (no visible desde afuera)
function area(a: number, b?: number): number {
  return b !== undefined
    ? (a * b) / 2      // triángulo
    : Math.PI * a * a  // círculo
}

area(5)      // → círculo (r=5)
area(3, 4)   // → triángulo (base=3, altura=4)
```

Contraste Kotlin — sobrecarga directa:

```
fun area(radio: Double): Double = Math.PI * radio * radio
fun area(base: Double, altura: Double): Double = base * altura / 2
```

Distinción: sobrecarga **no es** polimorfismo paramétrico. Son múltiples funciones distintas con el mismo nombre — el compilador elige cuál llamar. El binding es estático.

4.40 Polimorfismo paramétrico — Generics en TypeScript [F-43]

 **Louden §8.9 (Explicit Polymorphism):** “Parametric polymorphism is the ability for a function to be applied to arguments of different types without changing the code” — (Cita respaldada por ChromaDB, relevancia 0.767.)

```
// Función genérica — T es el parámetro de tipo
function primero<T>(lista: T[]): T { return lista[0] }

primero([1, 2, 3])      // → 1 (T inferred como number)
primero(["a", "b", "c"]) // → "a" (T inferred como string)

// Con upper bound — T debe tener valueOf()
function max<T extends { valueOf(): number }>(a: T, b: T): T {
  return a.valueOf() > b.valueOf() ? a : b
}

// Clase genérica
class Caja<T> {
  constructor(private contenido: T) {}
  abrir(): T { return this.contenido }
}

// Tipo genérico
type ReadOnlyBox<T> = { readonly value: T }
```

El tipo es un **parámetro** de la función o estructura — una sola implementación funciona para cualquier T.

4.41 Polimorfismo por subtipo [F-44]

 **Gabbrielli §8.8:** “By virtue of subtype compatibility, foo can receive as an argument a value of any subclass of A. The code for foo does not need to be adapted to specific subclasses” — (Cita respaldada por ChromaDB, relevancia 0.738.)

Un tipo S es subtipo de T si S puede usarse donde se espera T (*Principio de Liskov*).

```
interface Forma { area(): number }

class Circulo implements Forma {
  constructor(private r: number) {}
  area() { return Math.PI * this.r ** 2 }
}
class Rectangulo implements Forma {
  constructor(private b: number, private h: number) {}
  area() { return this.b * this.h }
}

function areaTotal(formas: Forma[]) {
  return formas.reduce((sum, f) => sum + f.area(), 0)
}

areaTotal([new Circulo(3), new Rectangulo(4, 5)]) // polimorfismo en acción
```

La diferencia con paramétrico: aquí el dispatch ocurre en **runtime** — el motor llama al método correcto según el tipo real del objeto. Con paramétrico, el binding es en compilación.

4.42 Comparación de los tres polimorfismos [F-45]

Aspecto	Ad-hoc (sobrecarga)	Paramétrico (generics)	Por subtipo (herencia)
Mecanismo	Nombre compartido	Parámetro de tipo <code><T></code>	Herencia / interface
Binding	Compilación (static dispatch)	Compilación	Runtime (dynamic dispatch)
Overhead	Ninguno	Ninguno (erasure en JVM)	Virtual dispatch
Restricción	Firmas distintas	Upper bounds (extends)	Is-a relationship
Ejemplo TS	Function overloads	<code>function f<T>(...)</code>	<code>class C implements I</code>
Expresividad	Baja	Alta	Media-Alta

La elección entre los tres depende del problema. Para casos limitados y específicos, sobrecarga. Para algoritmos genéricos, generics. Para jerarquías de objetos, subtipo. En la práctica, los tres conviven en lenguajes modernos.

4.43 Type Checking — definición y coerción [F-46]

📖 **Sebesta §6.13 (Type Checking):** “Type checking is the activity of ensuring that the operands of an operator are of compatible types” — (Cita respaldada por ChromaDB, relevancia 0.657.)
Gabbrielli §8.8 (Type Checking and Inference) — relevancia 0.723.

Tipo compatible: el tipo exacto esperado, o un tipo convertible implícitamente (→ **coerción**).

```
int i = 5;
float f = i + 3.14; // coerción implícita: int → float antes de sumar
float g = (float) i; // conversión explícita (casting)
```

Widening vs. Narrowing:

```
int → long → float → double    (widening — sin pérdida de información)
double → float → int             (narrowing — pérdida posible → warning)
```

Type error: aplicación de un operador a un operando de tipo **inapropiado**.

Estático vs. dinámico (Sebesta §6.13):

- Si todos los bindings son estáticos → chequeo **estático** (compilación).
- Binding dinámico → chequeo **dinámico** (runtime).
- TypeScript: chequeo estático encima de JavaScript → el runtime sigue siendo JS (type erasure).
- JavaScript/Python: solo dinámico por diseño.

“If all bindings of variables to types are static in a language, then type checking can nearly always be done statically” — Sebesta §6.13.

4.44 Equivalencia nominal vs. estructural [F-47]

 **Sebesta §6.15 (Type Equivalence):** *“There are two approaches to defining type equivalence: name type equivalence and structure type equivalence”* — (Cita respaldada por ChromaDB, relevancia 0.693.) **Gabbrielli §8.5:** *“Under equivalence by name, each type has a unique definition... equivalence by name is the choice that most respects the intentions of the designer”* — relevancia 0.656.

Equivalencia nominal: dos tipos son iguales si tienen el **mismo nombre**.

```
data class Celsius(val v: Double)
data class Fahrenheit(val v: Double)
// No son intercambiables — aunque ambas wrappean Double
fun calenta(c: Celsius) { ... }
// calenta(Fahrenheit(100.0)) ← ERROR de compilación ✓
```

Equivalencia estructural: dos tipos son iguales si tienen la **misma estructura**.

```
type Punto = { x: number; y: number }
// Cualquier objeto con campos x: number, y: number es compatible
function distancia(p: Punto) { ... }
distancia({ x: 3, y: 4 }) // ✓ — tipado estructural
```

 **Sebesta §6.15** advierte el riesgo canónico: `type Celsius = Float` y `type Fahrenheit = Float` en Ada serían equivalentes aunque semánticamente distintos. Esa es la debilidad del tipado estructural.

TypeScript es estructural — ventajas y riesgos:

- **Ventaja:** flexibilidad — menos boilerplate, duck typing seguro.
- **Riesgo:** `Celsius = number` y `Fahrenheit = number` serían equivalentes → usar **branded types** (`type Celsius = number & { readonly _brand: 'celsius' }`).

4.45 Equivalencia — tabla comparativa de lenguajes [F-48]

Lenguaje	Sistema	Observación
C (struct)	Nominal (con <code>typedef</code>)	Dos <code>struct</code> distintos → tipos distintos
Java	Nominal	<code>class A {}</code> y <code>class B {}</code> nunca son iguales
Kotlin	Nominal	Herencia/interfaces para compatibilidad
TypeScript	Estructural	Si la forma coincide, el tipo es compatible
Go	Estructural	Interfaces satisfechas implícitamente
Haskell	Nominal + paramétrico	<code>data</code> define tipos nominales

La equivalencia de tipos define cuándo el compilador **acepta** que un valor de un tipo sea usado donde se esperaba otro. Esa decisión afecta la flexibilidad, la seguridad semántica y la complejidad del compilador.

Bloque 5 — Síntesis, Discusión y Cierre (30 min) · Filminas F-49 a F-53

4.46 Mapa conceptual integrador [F-49]

Tipos de Datos:

- **Primitivos:** enteros · float (IEEE 754) · boolean · char
- **Ordinales:** enumeraciones · subrangos
- **Compuestos:** array · registro · tupla · unión discriminada · lista
- **Recursivos:** árbol · lista enlazada · ADT
- **Opcionales:** `T | null` · `Maybe a` · `Option<T>`

Sistemas de Tipos:

- **Dimensiones:** estático/dinámico · strong/weak · mono/polimórfico
- **Equivalencia:** nominal (Kotlin, Java) · estructural (TypeScript, Go)
- **Polimorfismo:** ad-hoc · paramétrico · subtipo
- **Null safety:** typed nulls → compile-time guarantees

4.47 Conexión con próximos temas [F-50]

- **T11 — Expresiones y Estructuras de Control:** coerciones implícitas en expresiones aritméticas, sobrecarga de operadores (polimorfismo ad-hoc en operadores), short-circuit evaluation y su relación con tipos booleanos.
- **T14 — Sistemas de Tipos Formales (futuro):** inferencia de tipos (algoritmo Hindley-Milner), λ -cálculo tipado como base formal, polimorfismo let (Milner) y sus límites, subtyping formal con reglas de deducción.

Recordar el vínculo con T09: “El sistema de tipos es la suma de las decisiones de binding que vimos en T09”. Un tipo primitivo, estáticamente vinculado, con equivalencia nominal = exactamente lo que vimos en la 5-tupla de variables.

4.48 Consigna del TP [F-51]

Objetivo: explorar el sistema de tipos de un lenguaje **no visto en profundidad en clase** y compararlo con TypeScript.

Consigna:

1. **Elegir** un lenguaje: Rust, Swift, Scala, Elm, Zig, Nim u otro aprobado.
2. **Documentar** al menos 5 aspectos del sistema de tipos: equivalencia (nominal/estructural), null safety (o ausencia), polimorfismo (tipos disponibles), strong/weak typing, tipo de GC o gestión de memoria.
3. **Comparar** con TypeScript usando ejemplos de código propios.
4. **Evaluar:** ¿qué hace mejor? ¿qué sacrifica? ¿qué decisión de diseño te parece más acertada?

Entrega: repositorio Git con código de ejemplo + informe `.md`. Fecha: ver cronograma en la plataforma.

Nota: las preguntas de autoevaluación de esta guía (sección 7) son **distintas** al TP — no duplican consignas.

4.49 Cierre [F-52]

- Taxonomía completa de tipos: primitivos → compuestos → recursivos → opcionales.
- Sistemas de tipos como espacio de decisiones de diseño.
- TypeScript como ejemplo de sistema de tipos moderno, pragmático y estructural.

“Un lenguaje de programación es solo tan bueno como su sistema de tipos.” — Bjarne Stroustrup

4.50 Q&A — tres preguntas para reflexionar [F-53]

1. **¿Por qué TypeScript hace `strictNullChecks` opt-in mientras Kotlin lo tiene activado siempre?** *Pista:* retrocompatibilidad con JavaScript, adopción gradual, codebase existente. Es una decisión de adopción masiva, no de teoría.
2. **¿Cuándo usar uniones discriminadas (TypeScript) vs. sealed class (Kotlin) vs. herencia tradicional?** *Pista:* exhaustividad, extensibilidad (Expression Problem), overhead de objetos. TypeScript: más conciso para tipos simples. Kotlin: exhaustividad automática. Herencia tradicional: cuando necesitás extensión abierta (Open/Closed Principle).
3. **¿Qué pierde TypeScript al tener tipado estructural en lugar de nominal?** *Pista:* `UserId` y `Email` como strings, `Celsius` y `Fahrenheit` como numbers — son intercambiables sin branded types. Una librería de terceros puede satisfacer accidentalmente una interfaz sin intención.

5. Ejemplos trabajados

Ejemplo 1 — Clasificar tipos primitivos vs. compuestos (OA2, OA6)

Consigna: Dadas las siguientes declaraciones en TypeScript, clasifica cada tipo según la taxonomía de la filmina F-05 (primitivo / definido por el usuario / compuesto / referencial / algebraico / paramétrico) y justifica.

```
const edad: number = 25
const inicial: string = "M"
const activo: boolean = true
const colores: [number, number, number] = [255, 128, 0]
const config: Record<string, boolean> = { dark: true }
const ptr: Int32Array = new Int32Array([1, 2, 3])
type Dia = 1 | 2 | 3 | 4 | 5
const hoy: Dia = 3
```

Resolución paso a paso:

Declaración	Tipo	Clasificación	Justificación
<code>edad: number</code>	<code>number</code>	Primitivo	Numérico, provisto por el lenguaje. En TS es IEEE 754 64-bit.
<code>inicial: string</code>	<code>string</code>	Primitivo	Texto elemental. En TS no existe <code>char</code> , así que incluso un carácter es <code>string</code> .
<code>activo: boolean</code>	<code>boolean</code>	Primitivo	Lógico, dominio <code>{true, false}</code> .
<code>colores: [number, number, number]</code>	tupla	Compuesto (producto) + Algebraico	Producto cartesiano <code>number × number × number</code> .
<code>config: Record<string, boolean></code>	<code>Record</code>	Compuesto (mapeo)	Array asociativo: claves <code>string</code> → valores <code>boolean</code> .
<code>ptr: Int32Array</code>	<code>Int32Array</code>	Compuesto (secuencia) + Paramétrico implícito	TypedArray: secuencia binaria de enteros de 32 bits.
<code>hoy: Dia</code>	<code>Dia = 1 2 3 4 5</code>	Definido por el usuario (subrango simulado via literal union)	Restringe el dominio de <code>number</code> a 5 valores.

 **Trampa común:** `string` en TypeScript es primitivo aunque tenga métodos (`.toUpperCase()`). El runtime hace autoboxing — no es un objeto. En Java sí sería un objeto.

Ejemplo 2 — Acceso a un array multidimensional (OA4)

Consigna: Tenés una matriz `int a[3][4]` en C (row-major, `int` de 4 bytes). La dirección base del array es `0x1000`. Calculá la dirección del elemento `a[2][3]`.

Resolución paso a paso:

1. Identificar los parámetros:

- `base = 0x1000 = 4096` (decimal)
- `cols = 4` (la matriz tiene 4 columnas)
- `size = 4` bytes (tamaño de `int`)
- `i = 2`, `j = 3`

2. Aplicar la fórmula row-major (filmina F-18):

```
dirección = base + (i × cols + j) × size
```

3. Sustituir:

```
dirección = 4096 + (2 × 4 + 3) × 4
           = 4096 + (8 + 3) × 4
           = 4096 + 11 × 4
           = 4096 + 44
           = 4140
           = 0x102C
```

4. Verificación conceptual: el elemento `a[2][3]` es el último de la fila 2. En row-major, los elementos de la fila 2 están en posiciones `(2×4 + 0)`, `(2×4 + 1)`, `(2×4 + 2)`, `(2×4 + 3)` = offsets 8, 9, 10, 11 desde el base. El offset `11 × 4 bytes = 44 bytes`. ✓

⚠ **Si la matriz fuera column-major (Fortran):** la fórmula sería `base + (i + j × rows) × size`. Con `rows = 3`: `4096 + (2 + 3 × 3) × 4 = 4096 + 11 × 4 = 4140`. En este caso particular coincide, pero en general **no** coinciden — por eso es crítico saber qué convención usa el lenguaje.

Ejemplo 3 — Análisis de un dangling pointer (OA8)

Consigna: Analizá el siguiente código en C y explicá qué problema tiene, por qué es peligroso, y cómo lo evitarías en TypeScript.

```
#include <stdlib.h>
int* crearYDevolver() {
    int* p = malloc(sizeof(int));
    *p = 42;
    free(p);
    return p;    // ← ¿qué pasa acá?
}

int main() {
    int* q = crearYDevolver();
    printf("%d\n", *q);    // ← ¿y acá?
    return 0;
}
```

Resolución paso a paso:

1. **Identificar el problema:** la función `crearYDevolver` asigna memoria con `malloc`, la libera con `free`, y **devuelve el puntero** a esa memoria ya liberada. El puntero `q` en `main` es un **dangling pointer** (puntero colgante).
2. **¿Por qué es peligroso?** (filmina F-32):
 - Después de `free(p)`, la memoria puede ser reutilizada por otra llamada a `malloc`.
 - `*q` lee memoria que ya no pertenece al programa — el valor puede ser basura, o puede ser un valor que otra parte del programa escribió.
 - El programa **no lanza error** — continúa con datos corruptos. Esto es lo que hace a los dangling pointers tan difíciles de debuggear.
 -  **Gabbrielli §8.5:** “If a language allows dangling references to happen, it is obvious that it cannot be type safe”.
3. **¿Cómo lo evito en TypeScript?** TypeScript no tiene `malloc` / `free` — el GC gestiona la memoria automáticamente. El equivalente sería:

```
function crearYDevolver(): { valor: number } {
    return { valor: 42 }    // el objeto vive en heap, gestionado por GC
}

const q = crearYDevolver()
console.log(q.valor)    // 42 — siempre válido
```

El GC libera el objeto solo cuando **ninguna** referencia lo apunta. Mientras `q` exista, el objeto no se libera. No hay dangling pointers posibles.

4. **¿Y en Rust?** Rust previene esto en compilación con el borrow checker: el código equivalente no compilaría porque `p` fue movido/liberado y no se puede devolver.

Ejemplo 4 — Equivalencia nominal vs. estructural (OA10)

Consigna: Dado el siguiente código, decid si compila en TypeScript y en Kotlin, y explicá por qué.

```
// TypeScript
type Punto2D = { x: number; y: number }
type Coordenada = { x: number; y: number }

const p: Punto2D = { x: 3, y: 4 }
const c: Coordenada = p // ← ¿compila?
```

```
// Kotlin
data class Punto2D(val x: Double, val y: Double)
data class Coordenada(val x: Double, val y: Double)

val p: Punto2D = Punto2D(3.0, 4.0)
val c: Coordenada = p // ← ¿compila?
```

Resolución paso a paso:

1. **TypeScript — SÍ compila.** TypeScript usa **equivalencia estructural** (filmina F-47). `Punto2D` y `Coordenada` tienen la misma estructura (`{ x: number; y: number }`), por lo tanto son equivalentes. El compilador acepta asignar `p` a `c`.

 **Sebesta §6.15:** “Structure type equivalence is more flexible than name type equivalence, but it is more difficult to implement”.

2. **Kotlin — NO compila.** Kotlin usa **equivalencia nominal** (filmina F-48). `Punto2D` y `Coordenada` son tipos distintos aunque tengan los mismos campos. El compilador rechaza la asignación con un error de tipo.

 **Gabrielli §8.5:** “equivalence by name is the choice that most respects the intentions of the designer”.

3. **Consecuencia práctica:** en TypeScript, si tenés `type UserId = string` y `type Email = string`, son intercambiables — el compilador no te avisa si los confundís. La solución son los **branded types**:

```
type UserId = string & { readonly _brand: 'userId' }
type Email = string & { readonly _brand: 'email' }
// Ahora no son intercambiables — estructuralmente distintos
```

Ejemplo 5 — Función genérica en TypeScript (OA11)

Consigna: Escribí una función genérica `longest<T>` que reciba dos arrays del mismo tipo y devuelva el más largo. Restringí `T` para que los elementos sean comparables con `<` (es decir, que tengan un método `valueOf(): number`). Mostrá cómo se invocaría con números y con strings.

Resolución paso a paso:

1. Identificar los requisitos:

- Función genérica con parámetro de tipo `T`.
- Recibe dos arrays `T[]`.
- Devuelve `T[]` (el más largo).
- Restricción: `T` debe tener `valueOf(): number` (upper bound con `extends`).

2. Escribir la firma:

```
function longest<T extends { valueOf(): number }>(a: T[], b: T[]): T[] {  
    return a.length >= b.length ? a : b  
}
```

Nota: el upper bound `{ valueOf(): number }` es **estructural** — cualquier tipo que tenga ese método califica, sin necesidad de implementar una interfaz nominal. Esto es tipado estructural en acción (filmina F-47).

3. Invocación con números:

```
const nums1: number[] = [1, 2, 3]  
const nums2: number[] = [4, 5]  
const masNums = longest(nums1, nums2) // → [1, 2, 3] (T = number)
```

`number` tiene `valueOf(): number` — satisface el constraint.

4. Invocación con strings:

```
const s1: string[] = ["a", "b"]  
const s2: string[] = ["x", "y", "z", "w"]  
const masStrings = longest(s1, s2) // → ["x", "y", "z", "w"] (T = string)
```

`string` también tiene `valueOf(): number` — satisface el constraint.

5. Invocación que NO compila (verificación del constraint):

```
const objs: { nombre: string }[] = [{ nombre: "a" }]  
// longest(objs, objs) // ❌ Error: { nombre: string } no tiene valueOf(): number
```

6. Contraste con polimorfismo por subtipo (filmina F-44):

si en lugar de generics usáramos una interfaz `interface Valuable { valueOf(): number }` y exigiéramos `Valuable[]`, el dispatch sería estructural pero la firma sería menos flexible — no aceptaría tipos que no declaren explícitamente implementar la interfaz. Con generics + upper bound estructural, cualquier tipo que tenga la forma correcta califica automáticamente.

 **Louden §8.9:** “Parametric polymorphism is the ability for a function to be applied to arguments of different types without changing the code” — una sola implementación de `longest` funciona para cualquier `T` que satisfaga el constraint.

6. Puntos clave y resumen (cheat-sheet)

Clasificación de tipos (filmina F-05)

Tipos

|— Primitivos: int, float, bool, char

|— Definidos por el usuario: enum, subrange, alias, branded

|— Compuestos: array, record, tuple, map, union

|— Referenciales: pointer, reference

|— Algebraicos: producto (record), suma (union/ADT)

|— Paramétricos: generics, templates

Primitivos vs. ordinales vs. compuestos

Categoría	Definición	Ejemplos TS
Primitivos	Provistos por el lenguaje/plataforma	number , bigint , boolean , string
Ordinales	Discretos, enumerables, con sucesor/predecesor	boolean , enum , subrangos simulados con literal unions
Compuestos	Construyen valores más grandes desde otros	T[] , interface , [A, B] , Map<K,V> , uniones discriminadas

Arrays rectangulares vs. jagged

Tipo	Característica	Lenguajes
Rectangular	Sub-arrays de igual longitud, acceso lineal	Fortran, C# (int[,])
Jagged	Array de punteros a arrays, filas de distinto tamaño	C, Java, Kotlin, TypeScript

Punteros vs. referencias

Aspecto	Puntero C	Referencia (C++)	Referencia TS/Kotlin
Aritmética	✓	✗	✗
Reasignable	✓	✗	✓
Puede ser null	✓	✗	✓ (con tipo)
Gestión memoria	Manual	Manual/RAII	GC

Type checking

Dimensión	Valores
Cuándo verifica	Estático (compilación) vs. dinámico (runtime)
Coerción	Implícita (compilador) vs. explícita (casting)
Dirección	Widening (seguro) vs. narrowing (pérdida)

Equivalencia de tipos

Sistema	Criterio	Lenguajes
Nominal	Mismo nombre	C (struct), Java, Kotlin, Haskell
Estructural	Misma estructura	TypeScript, Go

Polimorfismo (taxonomía Strachey/Cardelli)

Polimorfismo

- Ad-hoc (apariencia)
 - Sobrecarga (overloading)
 - Coerción
- Universal (uniformidad real)
 - Paramétrico (generics)
 - Subtipo / Inclusión (herencia)

Null safety — operadores TypeScript

Operador	Nombre	Equivalente Kotlin
<code>T null</code>	Tipo nutable	<code>T?</code>
<code>?.</code>	Optional chaining	<code>?.</code>
<code>??</code>	Nullish coalescing	<code>?:</code> (Elvis)
<code>!</code>	Non-null assertion	<code>!!</code>
<code>if (x !== null)</code>	Type narrowing	<code>if (x != null)</code>

7. Autoevaluación

Las siguientes preguntas son **distintas** a las del TP — sirven para verificar tu comprensión del tema. Las respuestas están en `<details>` para que primero intentes resolverlas.

Nivel Recordar / Comprender

P1. Según Sebesta §6.1, ¿cuál es la definición formal de tipo de dato? Menciona las tres razones principales por las que los lenguajes incluyen tipos.

► Respuesta

P2. ¿Por qué `0.1 + 0.2 !== 0.3` en TypeScript? ¿Este problema es exclusivo de TypeScript?

► Respuesta

P3. ¿Qué es un tipo ordinal? Da dos ejemplos que no sean `boolean`.

► Respuesta

P4. ¿Por qué TypeScript no tiene tipo `char`? ¿Cómo se representa un carácter individual?

► Respuesta

Nivel Aplicar

P5. Reescribí el siguiente código JavaScript (sin `strictNullChecks`) usando TypeScript con null safety. Usá `?.`, `??` y type narrowing donde corresponda.

```
function imprimirUsuario(user) {  
  const email = user.email  
  const dominio = email.split('@')[1]  
  console.log("Enviando a " + dominio.toUpperCase())  
}
```

Asumí que `user.email` puede ser `null` y que `user` siempre existe.

► Respuesta

P6. Dado un array `int a[5][10]` en C (row-major, `int` = 4 bytes, base = `0x2000`), calculá la dirección de `a[3][7]`.

► Respuesta

P7. Escribí una unión discriminada `type Estado` con tres variantes: `Cargando`, `Exito` (con `datos: string[]`), y `Error` (con `mensaje: string`). Escribí una función `manejar` que haga switch exhaustivo sobre `kind`.

► Respuesta

Nivel Analizar

P8. Compará `union` de C con discriminated unions de TypeScript. ¿Cuál es más seguro y por qué? Citá Sebesta.

► Respuesta

P9. ¿Cuál es la diferencia entre polimorfismo paramétrico y polimorfismo por subtipo? Da un ejemplo de cada uno en TypeScript y explica en qué momento (compilación vs. runtime) ocurre el dispatch.

► Respuesta

P10. Explicá por qué `type Celsius = number` y `type Fahrenheit = number` en TypeScript son un problema. ¿Qué solución propone el diseño de clase? ¿Cómo lo evita Kotlin?

► Respuesta

P11. Analizá la siguiente afirmación: *"TypeScript es fuertemente tipado"*. ¿Es verdadera, falsa o matizable? Justificá citando Sebesta §6.14.

► Respuesta

Nivel Evaluar

P12. Dado un programa TypeScript que usa `any` extensivamente, evaluá su nivel de tipado fuerte y proponé dos mejoras concretas.

► Respuesta

P13. ¿Por qué TypeScript hace `strictNullChecks` opt-in mientras Kotlin lo tiene activado siempre? Evaluá la decisión de diseño de cada lenguaje.

► Respuesta

P14. Dadas tres implementaciones de "resultado de operación" (C `union`, TypeScript discriminated union, Kotlin `sealed class`), evaluá cuál usarías para: (a) un sistema crítico donde la seguridad es prioritaria, (b) un prototipo rápido, © un sistema que debe extenderse con nuevas variantes sin recompilar.

► Respuesta

Nivel Sintetizar

P15. Construí un mapa conceptual que conecte: tipo primitivo, tipo compuesto, tipo recursivo, unión discriminada, tipo opcional, y sistema de tipos. Explicá las flechas.

► Respuesta

P16. ¿Qué relación hay entre el concepto de "binding de tipos" (T09) y la "equivalencia de tipos" (T10)? Sintetizá en un párrafo.

► Respuesta

P17. Un compañero dice: *"Python `list` es una lista enlazada porque se llama `list`"*. ¿Es correcto? Justificá con datos de la familia F-26 y conceptos de complejidad algorítmica.

► Respuesta

Nivel adicional (comprensión profunda)

P18. ¿Por qué la recursión de tipos es posible solo con punteros/referencias y no con el objeto completo? Usá el ejemplo de la lista enlazada en C.

► Respuesta

P19. Dado el siguiente código TypeScript, identificá qué tipo de polimorfismo usa cada función (`area` , `primero` , `areaTotal`) y justificá.

```
function area(radio: number): number
function area(base: number, altura: number): number
function area(a: number, b?: number): number { return b !== undefined ? a * b / 2 : Math.PI * a * a }

function primero<T>(lista: T[]): T { return lista[0] }

interface Forma { area(): number }
function areaTotal(formas: Forma[]): number { return formas.reduce((s, f) => s + f.area(), 0) }
```

► Respuesta

P20. Explicá la cita de Tony Hoare (“my billion-dollar mistake”) en relación con los tipos opcionales. ¿Cómo cambia el sistema de tipos cuando `null` deja de ser “un valor que cualquier referencia puede tener” y pasa a ser “un tipo que debe declararse explícitamente”?

► Respuesta

8. Glosario

Término	Definición
Tipo (data type)	Conjunto de valores + conjunto de operaciones predefinidas sobre esos valores (Sebesta §6.1).
Type system (sistema de tipos)	Conjunto de reglas que define cómo los tipos son asignados, verificados y combinados en un lenguaje (Louden §8.1).
Primitive (tipo primitivo)	Tipo básico provisto por el lenguaje o la plataforma: <code>int</code> , <code>float</code> , <code>bool</code> , <code>char</code> .
Ordinal	Tipo con valores discretos que pueden enumerarse y, en muchos lenguajes, ordenarse. Propiedades: sucesor/predecesor, comparación, rangos.
Enum (enumeración)	Tipo ordinal definido por el usuario que crea un dominio simbólico cerrado. En C no es type-safe; en TypeScript/Kotlin/Haskell sí.
Subrange (subrango)	Tipo cuyo dominio queda restringido a un rango de otro tipo ordinal. Pascal/Ada lo tienen nativo; TypeScript lo simula con literal unions.
Array	Tipo compuesto secuencia: colección ordenada de elementos del mismo tipo, accesibles por índice. Se clasifica por binding time de forma y rango.
Record (registro)	Tipo compuesto producto: colección de campos heterogéneos identificados por nombre. En TypeScript: <code>interface</code> / <code>type</code> .
Tuple (tupla)	Tipo compuesto producto: como un record pero los elementos no se identifican por nombre, sino por posición. Producto cartesiano $A \times B$.
Map (array asociativo)	Tipo compuesto mapeo: colección no ordenada de elementos indexada por claves. En TypeScript: <code>Map<K,V></code> o <code>Record<string, V></code> .
Union (unión libre)	Tipo suma donde distintos campos comparten el mismo espacio de memoria sin etiqueta discriminante. Inseguro en C — el programador debe recordar qué campo es válido.
Discriminated union (unión discriminada)	Tipo suma con etiqueta discriminante (tag) que indica qué variante es activa. Segura: el compilador verifica el acceso. TypeScript (<code>kind</code>), Kotlin (<code>sealed class</code>), Haskell (<code>data</code>).
Pointer (puntero)	Tipo referencial: valor que contiene una dirección de memoria + valor especial <code>nil</code> . Permite aritmética de punteros. C es el ejemplo canónico.
Reference (referencia)	Puntero constante implícitamente dereferenciado (Sebesta §6.11.5). Sin aritmética, no reasignable (en C++), gestionado por GC en TS/Kotlin/Java.
Dangling pointer (puntero colgante)	Puntero que apunta a memoria ya liberada. Problema clásico de C/C++. El GC lo elimina en lenguajes modernos.

Término	Definición
Type checking (chequeo de tipos)	Actividad de asegurar que los operandos de un operador son de tipos compatibles (Sebesta §6.13). Puede ser estático (compilación) o dinámico (runtime).
Strong typing (tipado fuerte)	Un lenguaje es strongly typed si todos los errores de tipo son siempre detectados (Sebesta §6.14). Haskell y Ada son los más strongly typed.
Weak typing (tipado débil)	El lenguaje permite operaciones entre tipos incompatibles sin detectar el error (o con coerciones implícitas abundantes). C es el ejemplo canónico.
Type equivalence (equivalencia de tipos)	Criterio que define cuándo dos tipos son considerados iguales. Nominal: mismo nombre. Estructural: misma estructura (Sebesta §6.15).
Name equivalence (equivalencia nominal)	Dos tipos son equivalentes si tienen el mismo nombre. Usada por Java, Kotlin, C (struct), Haskell.
Structural equivalence (equivalencia estructural)	Dos tipos son equivalentes si tienen la misma estructura. Usada por TypeScript, Go. Más flexible pero no distingue tipos con misma estructura y semántica distinta.
Polymorphism (polimorfismo)	Capacidad de una función o tipo de operar sobre múltiples tipos. Taxonomía Strachey/Cardelli: ad-hoc (sobrecarga, coerción) y universal (paramétrico, subtipo).
Parametric polymorphism (polimorfismo paramétrico)	Una implementación funciona para cualquier tipo <code>T</code> — el tipo es un parámetro. Implementado con generics. Binding en compilación, sin overhead.
Subtype polymorphism (polimorfismo por subtipo)	Un tipo <code>S</code> puede usarse donde se espera <code>T</code> si <code>S</code> es subtipo de <code>T</code> (Principio de Liskov). Dispatch en runtime (virtual dispatch).
Ad-hoc polymorphism (polimorfismo ad-hoc)	Mismo nombre, múltiples implementaciones por tipo (sobrecarga) o conversión implícita (coerción). Apariencia de uniformidad, no uniformidad real.
Generic (genérico)	Mecanismo de polimorfismo paramétrico: <code>function f<T>(...)</code> o <code>class Caja<T></code> . Una sola implementación para múltiples tipos.
Optional type (tipo opcional)	Tipo que permite representar explícitamente la ausencia de valor: <code>T null</code> (TypeScript), <code>T?</code> (Kotlin), <code>Maybe a</code> (Haskell), <code>Option<T></code> (Rust).
Coercion (coerción)	Conversión implícita de tipos generada por el compilador. <code>int + float</code> → el <code>int</code> es coercionado a <code>float</code> . Widening (seguro) vs. narrowing (pérdida posible).
Branded type (tipo marcado)	Técnica de TypeScript para distinguir tipos con la misma estructura: <code>type Celsius = number & { readonly _brand: 'celsius' }</code> . Simula equivalencia nominal sobre tipado estructural.

Término	Definición
TypedArray	Familia de tipos de TypeScript (<code>Int32Array</code> , <code>Float64Array</code> , etc.) que almacenan datos en binario puro en un <code>ArrayBuffer</code> , sin overhead de boxing. Para rendimiento y comunicación binaria.

9. Referencias y lecturas recomendadas

Bibliografía primaria

- **Sebesta, Robert W.** (2019). *Concepts of Programming Languages*, 12ª ed. Pearson.
 - **Cap. 6 — Data Types** (pp. 259–324): §6.1 Introduction, §6.2 Primitive Data Types (incl. §6.2.1.1 Integer, §6.2.1 Floating-point, §6.2.4 Boolean), §6.3 Character String Types, §6.4 Enumeration Types (§6.4.1), §6.5 Array Types (Figura 6.2), §6.6 Associative Arrays, §6.7 Record Types, §6.8 Tuple Types, §6.9 List Types, §6.10 Union Types, §6.11 Pointer and Reference Types (§6.11.1, §6.11.4 Tombstones/Locks, §6.11.5 Reference Types, §6.11.6 Dangling pointers), §6.12 Optional Types, §6.13 Type Checking, §6.14 Strong Typing, §6.15 Type Equivalence.
 - *Citas verificadas vía ChromaDB (relevancia ≥ 0.489).*

Bibliografía secundaria

- **Louden, Kenneth C. & Lambert, Kenneth A.** (2012). *Programming Languages: Principles and Practice*, 3ª ed. Cengage.
 - **Cap. 8 — Data Types and Type Information** (pp. 328–405): §8.1, §8.2 Simple Types, §8.5 Type Equivalence, §8.6 Type Checking, §8.8 Polymorphic Type Checking, §8.9 Explicit Polymorphism.
 - *Citas verificadas vía ChromaDB (relevancia ≥ 0.503).*
- **Gabbrielli, Maurizio & Martini, Simone** (2023). *Programming Languages: Principles and Paradigms*, 2ª ed. Springer.
 - **Cap. 8 — Structuring Data / Composite Types** (pp. 136–282): §8.2 Type Safety, §8.3.7 Enumerations, §8.3.9 Intervals, §8.4 Arrays, §8.4.3 Tagged Unions, §8.5 Dangling references, §8.7 Union types, §8.8 Type Checking and Inference, §8.8 Polymorphism.
 - *Citas verificadas vía ChromaDB (relevancia ≥ 0.516).*

Referencias de lenguaje (documentación oficial)

- TypeScript Handbook — Types: <https://www.typescriptlang.org/docs/handbook/2/types-from-types.html>
- TypeScript Handbook — Narrowing & Null: <https://www.typescriptlang.org/docs/handbook/2/narrowing.html>
- TypeScript Handbook — Generics: <https://www.typescriptlang.org/docs/handbook/2/generics.html>
- Kotlin Documentation — Null Safety: <https://kotlinlang.org/docs/null-safety.html>
- Kotlin Documentation — Generics (varianza): <https://kotlinlang.org/docs/generics.html>

Lectura opcional avanzada

- **Cardelli, Luca & Wegner, Peter** (1985). *“On understanding types, data abstraction, and polymorphism”*. ACM Computing Surveys 17(4). — Fuente de la taxonomía del polimorfismo usada en el Bloque 4.
- **Hoare, Tony** (2009). *“Null References: The Billion Dollar Mistake”*. QCon London. — Fuente de la cita del Bloque 3 sobre null safety.

Trazabilidad de citas por filmina (respaldo ChromaDB)

Filmina	Sección .txt	Respaldo bibliográfico (ChromaDB)
F-04	26-37	Sebesta §6.1 (relevancia 0.627)
F-07	77-122	Sebesta §6.2.1.1 (relevancia 0.594)
F-08	123-172	Sebesta §6.2.1 (relevancia 0.593)
F-10	199-218	Sebesta §6.2.4 (relevancia 0.627)
F-13	263-301	Sebesta §6.4.1 (relevancia 0.657)
F-14	303-362	Louden §8.2 (0.503), Gabbrielli §8.3.9 (0.529)
F-16	384-414	Sebesta §6.3 (relevancia 0.67)
F-17	415-447	Sebesta §6.5 (relevancia 0.492)
F-20	504-521	Sebesta §6.8 (relevancia 0.489)
F-23	565-585	Sebesta §6.7 (relevancia 0.492)
F-24	587-601	Sebesta §6.8 (relevancia 0.458)
F-26	616-665	Sebesta §6.9 (relevancia 0.492)
F-28	695-710	Sebesta §6.10 (relevancia 0.691)
F-30	745-751	Gabbrielli §8.4.3 (relevancia 0.682)
F-31	752-765	Sebesta §6.11.1 (relevancia 0.699)
F-32	766-779	Sebesta §6.11.6 (0.688), Gabbrielli §8.5 (0.735)
F-34	805-818	Sebesta §6.11.5 (relevancia 0.68)
F-37	982-987	Sebesta §6.12 (relevancia 0.613)
F-41	877-890	Louden §8.8 (0.767), Gabbrielli §8.8 (0.726)
F-43	911-931	Louden §8.9 (relevancia 0.767)
F-46	1047-1066	Sebesta §6.13 (0.657), Gabbrielli §8.8 (0.723)
F-47	1068-1095	Sebesta §6.15 (0.693), Gabbrielli §8.5 (0.656)

Nota sobre los PDFs fuente: los textos de Sebesta, Louden y Gabbrielli están ingestados en la base de conocimiento ChromaDB del proyecto (`_edu-knowledge/`). Las citas de esta guía fueron verificadas mediante queries a ChromaDB (`python scripts/knowledge_base.py search "..."` `--type material`). No se inventaron páginas ni secciones — todas las citas corresponden a fragmentos reales de los libros ingestados.

 *"Si un alumno puede estudiarlo solo, lo hicimos bien."*

Fin de la guía de estudio — Tema 10. Para profundizar, consultar las filminas (`filminas.md`), la minuta docente (`minuta.md`) y la bibliografía listada arriba. Las preguntas de la sección 7 son de autoevaluación y **no** duplican las consignas del TP.